অধ্যায় ২৪

২৪

JavaScript থেকে TypeScript: Static Typing, Type Safety এবং Professional Development Workflow

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- TypeScript কী নিশ্চিত করে এবং কী করে না—Compile-time Check ও Runtime Validation-এর পার্থক্য—স্পষ্টভাবে বলতে।
- `tsconfig.json` -এর Strict Setting দিয়ে একটি Project তৈরি করে `tsc` দিয়ে Type Check ও Build করতে।
- Type Inference, Literal Type, Union, `type` ও `interface` ব্যবহার করতে।
- `strictNullChecks` -এর অধীনে `undefined` ও `null` নিরাপদে সামলাতে।
- `unknown` দিয়ে Trust Boundary-তে তথ্য গ্রহণ করে Type Guard দিয়ে যাচাই করতে, এবং `any` ও `as` -এর ঝুঁকি চিনতে।
- Discriminated Union, `Result` Type ও `never` দিয়ে Exhaustive Check লিখতে।
- Generic Function ও Generic Class, এবং `keyof` , `Pick` , `Omit` , `Partial` , `Record` -এর মতো Utility Type ব্যবহার করতে।
- Branded Type দিয়ে একই ধরনের ভিন্ন অর্থের মান (টাকা বনাম সংখ্যা) আলাদা রাখতে।
- Compile-time Test (`@ts-expect-error`) ও Runtime Test দুটিই লিখতে।
- একটি সম্পূর্ণ **TypeSafe Claims** Service তৈরি করতে।

পূর্বশর্ত

- অধ্যায় ৪–৭: Function, Object, Array, Destructuring।
- অধ্যায় ১১ ও ১৭: Validation, Trust Boundary।
- অধ্যায় ১৮: Result Type, Immutable হালনাগাদ।
- অধ্যায় ২১: State Machine, Repository, Dependency Injection।
- অধ্যায় ২২: `node:test`।

এই অধ্যায় কেন গুরুত্বপূর্ণ

JavaScript-এ একটি Function যেকোনো মান গ্রহণ করে। `totalPrice(1500, '2')` কোনো আপত্তি ছাড়াই চলে, এবং ভুল ধরা পড়ে অনেক পরে—হয়তো গ্রাহকের Invoice-এ। দলে Code বড় হলে প্রশ্নগুলো জমতে থাকে: এই Function কী ফেরত দেয়? এই Field কি কখনো `undefined` হতে পারে? এই Status-এর তালিকায় নতুন একটি মান যোগ করলে কোন কোন `switch` হালনাগাদ করতে হবে?

TypeScript JavaScript-এর ওপর একটি Type-এর স্তর যোগ করে, এবং এই প্রশ্নগুলোর উত্তর Code চালানোর আগেই, Compiler-এর কাছ থেকে দেয়। আজ বেশিরভাগ বড় JavaScript Project—Framework, Library, Backend—TypeScript-এ লেখা। কিন্তু একটি ভুল ধারণাও ব্যাপক: "TypeScript থাকলে ইনপুট নিরাপদ"। তা নয়। TypeScript-এর সব Type Compile-এর পর মুছে যায়; Browser বা Server-এ যা চলে তা সাধারণ JavaScript। এই অধ্যায়ের কেন্দ্রীয় শিক্ষা তাই দুটি: Type দিয়ে আপনার নিজের Code-এর ভুল ধরুন, এবং বাইরে থেকে আসা তথ্য Runtime-এ যাচাই করুন।

২৪.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
`typeof`, Runtime যাচাই (অধ্যায় ১১, ১৭)	Type Guard: যাচাই, যা Compiler-কেও Type জানায়
Result Object (অধ্যায় ১৮)	Discriminated Union হিসেবে `Result<T, E>`
টেবিলভিত্তিক State Machine (অধ্যায় ২১)	`Record<Status, ...>` ও `never` দিয়ে সম্পূর্ণতার নিশ্চয়তা
Repository-র চুক্তি (অধ্যায় ২১)	Generic `Repository<T, K>` Interface
JSDoc মন্তব্যে Type বর্ণনা	Compiler-যাচাই করা Type
—	`tsconfig.json`, Branded Type, Compile-time Test

২৪.২ Dynamic Typing-এর একটি চেনা ভুল, এবং TypeScript-এর উত্তর

Form থেকে আসা প্রতিটি মান লেখা (String)। নিচের JavaScript নীরবে ভুল করে:

dynamic.js · JavaScript — চলবে: Node.js

```javascript
function totalPrice(price, quantity) {
  return price * quantity;
}

function addFee(total, fee) {
  return total + fee;
}

const quantityFromForm = '2';
const total = totalPrice(1500, quantityFromForm);
console.log(total, addFee(total, '50'));
```

Output

```
3000 300050
```

গুণ লেখাকে সংখ্যায় রূপান্তর করে "ঠিক" ফল দিয়েছে, কিন্তু যোগ লেখা জোড়া লাগিয়েছে: ৩,০৫০-এর বদলে ৩,০০,০৫০। একই Code TypeScript-এ, প্রতিটি Parameter-এর Type লিখে:

typed.ts · TypeScript — চলবে: Node.js

```typescript
function totalPrice(price: number, quantity: number): number {
  return price * quantity;
}

const quantityFromForm = '2';
console.log(totalPrice(1500, quantityFromForm));
```

`tsc --strict` চালালে:

Error Message

```
error TS2345: Argument of type 'string' is not assignable to parameter
of type 'number'.
```

ভুলটি Program চালানোর আগেই ধরা পড়েছে, এবং ঠিক কোন লাইনে তা বলা হয়েছে। সমাধানও স্পষ্ট হয়ে যায়: Form-এর লেখাকে সংখ্যায় রূপান্তর ও যাচাই করা— `Number(quantityFromForm)`, এবং `Number.isInteger` দিয়ে পরীক্ষা।

TypeScript কাজ করে এভাবে: আপনি `.ts` File লেখেন, Compiler (`tsc`) Type যাচাই করে, তারপর Type মুছে সাধারণ JavaScript তৈরি করে (**Type Erasure**)। Node.js ২২.৬ থেকে `--experimental-strip-types` দিয়ে `.ts` File সরাসরি চালানো যায়, কিন্তু সেটি শুধু Type **মুছে ফেলে**, যাচাই করে না—Type Check-এর জন্য `tsc` লাগবেই।

২৪.৩ Project-এর কাঠামো এবং `tsconfig.json`

একটি TypeScript Project-এ সাধারণত TypeScript নিজে (এবং Node.js-এর Type-এর জন্য `@types/node`) Development Dependency হিসেবে থাকে:

Terminal

```
npm install --save-dev typescript @types/node
npx tsc --init
```

> **নোট:** এই বইয়ের QA-তে কোনো Package Install করা হয়নি। এই কম্পিউটারে আগে থেকে Install করা একটি Global Tool-এর ভেতরে TypeScript 6.0.2 ও `@types/node` 25.6.0 পাওয়া গেছে; এই অধ্যায়ের সব Compile ও Test সেগুলো দিয়ে চালানো। আপনার Project-এ উপরের Command দিয়ে নিজস্ব Copy Install করুন।

Project-এর `tsconfig.json`-এ গুরুত্বপূর্ণ Setting:

tsconfig.json · JSON — চলবে: Node.js

```json
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": false,
    "verbatimModuleSyntax": true,
    "types": ["node"],
    "rootDir": ".",
    "outDir": "dist",
    "sourceMap": true
  },
  "include": ["src", "tests"]
}
```

Setting	কাজ
`strict`	সব কঠোর Check একসঙ্গে, এর মধ্যে `strictNullChecks` ও `noImplicitAny`; নতুন Project-এ সব সময় চালু
`noUncheckedIndexedAccess`	`array[i]`-এর Type `T` অথবা `undefined`, কারণ Index-এ মান না-ও থাকতে পারে
`exactOptionalPropertyTypes`	ঐচ্ছিক Field-এ `undefined` স্পষ্টভাবে লেখা আর অনুপস্থিত থাকা আলাদা
`module` ও `moduleResolution`: `NodeNext`	Node.js-এর আসল ES Module নিয়ম; তাই Import-এ `.js` লেখা হয়
`verbatimModuleSyntax`	শুধু Type-এর Import (`import type`) Output থেকে মুছে যায়, বাকিগুলো হুবহু থাকে
`outDir`	Compile করা JavaScript `dist/`-এ

Import-এ `.js` প্রথমে অদ্ভুত লাগে (`import { toPaisa } from './money.js'`, অথচ File-টি `money.ts`), কিন্তু যুক্তিটি সরল: Node.js চালাবে Compile-এর পরের `money.js`, এবং TypeScript Import-এর পথ বদলায় না।

২৪.৪ Type Inference, Literal Type এবং Union

প্রতিটি Variable-এ Type লিখতে হয় না; TypeScript মান দেখে **অনুমান** (Infer) করে। Type লিখুন সীমানায়—Function-এর Parameter ও ফেরত মান, এবং Export করা জিনিসে।

inference.ts · TypeScript — চলবে: Node.js

```ts
let seats = 40;
const mode = 'bus';

type Mode = 'bus' | 'train' | 'air';
function fare(travelMode: Mode, km: number): number {
  const perKm = { bus: 2, train: 1.5, air: 9 }[travelMode];
  return Math.round(perKm * km);
}

seats = 'forty';
console.log(fare(mode, 220), fare('ship', 100));
```

Error Message

```
error TS2322: Type 'string' is not assignable to type 'number'.
error TS2345: Argument of type '"ship"' is not assignable to parameter
of type 'Mode'.
```

দুটি পর্যবেক্ষণ। `let seats = 40`-এর Type অনুমিত `number`, তাই পরে লেখা বসানো ভুল। আর `const mode = 'bus'`-এর Type শুধু `string` নয়, **Literal Type** `'bus'` — `const` কখনো বদলায় না—তাই এটি `Mode`-এ খাপ খায়। `Mode` একটি **Union**: তিনটির যেকোনো একটি, এর বাইরে কিছু নয়। Status, Mode, Category-র মতো সীমিত মানের তালিকার জন্য Union-ই সবচেয়ে নিরাপদ—বানান ভুল (`'trian'`) Compile-এই ধরা পড়ে।

`type` বনাম `interface`: দুটিই Object-এর আকার বর্ণনা করে। `interface` প্রসারিত (`extends`) ও পরে আবার ঘোষণা করে মেলানো যায়; `type` Union, Tuple ও জটিল রূপান্তরের জন্য। একটি সহজ নিয়ম: Object-এর চুক্তির জন্য `interface`, বাকি সবকিছুর জন্য `type`—এবং একটি Project-এ ধারাবাহিক থাকুন।

২৪.৫ null, undefined এবং Strict Mode

`strict`-এর সবচেয়ে মূল্যবান অংশ `strictNullChecks`: যে মান `undefined` হতে পারে, তাকে যাচাই না করে ব্যবহার করা যায় না। JavaScript-এর সবচেয়ে সাধারণ Runtime Error—"Cannot read properties of undefined"—এর বড় অংশ এভাবেই Compile-এ ধরা পড়ে।

nulls.ts · TypeScript — চলবে: Node.js

```
interface Traveler {
  name: string;
  passport?: string;
}

const travelers: Traveler[] = [{ name: 'Rafi', passport: 'BX1234567' }];
const found = travelers.find((t) => t.name === 'Mina');
console.log(found.name.toUpperCase());
console.log(travelers[0].passport.slice(0, 2));
```

Error Message

```
error TS18048: 'found' is possibly 'undefined'.
error TS2532: Object is possibly 'undefined'.
```

`find` কিছু না পেলে `undefined` দেয়, আর `passport?` ঐচ্ছিক। সমাধান—যাচাই, Optional Chaining বা Default মান:

nulls-fixed.ts · TypeScript — চলবে: Node.js

```ts
interface Traveler {
  name: string;
  passport?: string;
}

const travelers: Traveler[] = [{ name: 'Rafi', passport: 'BX1234567' }];
const found = travelers.find((t) => t.name === 'Mina');
if (found) {
  console.log(found.name.toUpperCase());
}
console.log(found?.name ?? 'Mina নামে কেউ নেই');
console.log(travelers[0]?.passport?.slice(0, 2) ?? 'Passport নেই');
```

Output

```
Mina নামে কেউ নেই
BX
```

`if (found)` -এর ভেতরে TypeScript জানে `found` আর `undefined` নয়—একে বলে **Narrowing**। Non-null Assertion (`found!.name`) Compiler-কে চুপ করায়, কিন্তু Runtime-এ কিছুই যাচাই করে না; তাই এটি প্রায় সব সময় এড়িয়ে চলুন।

২৪.৬ unknown বনাম any: Trust Boundary

বাইরের তথ্য— `JSON.parse` -এর ফল, `fetch` -এর Body, Form, File—Compile-এর সময় অজানা। TypeScript দুটি Type দেয়, যার আচরণ বিপরীত:

- **`any`** : Type Check বন্ধ। যেকোনো কিছু করা যায়, এবং ভুল Runtime-এ পর্যন্ত লুকিয়ে থাকে।
- **`unknown`** : "জানি না"। যাচাই (Narrowing) না করে কিছুই করা যায় না।

any.ts · TypeScript — চলবে: Node.js

```ts
const text = '{ "name": 42 }';
const data: any = JSON.parse(text);

try {
  console.log(data.name.toUpperCase());
} catch (error) {
  console.log(`Runtime-এ ধরা পড়ল: ${(error as Error).message}`);
}
```

Output

```
Runtime-এ ধরা পড়ল: data.name.toUpperCase is not a function
```

`any`-এর সঙ্গে Compiler কোনো আপত্তি করেনি; ভুলটি ধরা পড়েছে চলার সময়। একই কাজ `unknown`-এ:

unknown.ts · TypeScript — চলবে: Node.js

```ts
const data: unknown = JSON.parse('{ "name": 42 }');
console.log(data.name.toUpperCase());
```

Error Message

```
error TS18046: 'data' is of type 'unknown'.
```

এখন Compiler যাচাই দাবি করছে। যাচাইয়ের সেরা হাতিয়ার **Type Guard**—একটি Function, যার ফেরত Type `value is T`: Runtime-এ পরীক্ষা করে, এবং `true` হলে Compiler-কেও Type জানায়:

guard.ts · TypeScript — চলবে: Node.js

```ts
interface Customer {
  name: string;
  phone: string;
}

function isCustomer(value: unknown): value is Customer {
  if (typeof value !== 'object' || value === null) {
    return false;
  }
  const record = value as Record<string, unknown>;
  return typeof record.name === 'string' && typeof record.phone === 'string';
}

for (const text of ['{ "name": "Rafi", "phone": "017" }', '{ "name": 42 }']) {
  const data: unknown = JSON.parse(text);
  console.log(isCustomer(data) ? data.name.toUpperCase() : 'অবৈধ তথ্য');
}
```

Output

```
RAFI
অবৈধ তথ্য
```

`as` (Type Assertion) সবচেয়ে বিপজ্জনক: `JSON.parse(text) as Customer` Compiler-কে বলে "বিশ্বাস করো", এবং Compiler বিশ্বাস করে—কোনো যাচাই ছাড়াই। অধ্যায় ২০-এর নিয়ম এখানেও: **TypeScript Runtime-এ ইনপুটের নিরাপত্তা দেয় না।** Server-এ পৌঁছানো প্রতিটি তথ্য Runtime-এ যাচাই করতে হবে; TypeScript শুধু নিশ্চিত করে

যে যাচাইয়ের পর আপনার Code সঠিক Type ব্যবহার করছে। বড় Project-এ এই যাচাইয়ের জন্য Zod বা Valibot-এর মতো Library একবার লেখা Schema থেকে Validation ও Type দুটিই তৈরি করে।

২৪.৭ Discriminated Union, `Result` এবং Exhaustiveness

একটি Union-এর প্রতিটি সদস্যে একটি সাধারণ Literal Field (Discriminant, যেমন `kind` বা `ok`) থাকলে, সেই Field দেখে TypeScript বাকি Field জানে। Error-এর জন্য Exception-এর বদলে একটি `Result` Type (অধ্যায় ১৮) এখন Compiler-যাচাই করা:

result.ts · TypeScript — চলবে: Node.js

```ts
type Result<T, E> = { ok: true; value: T } | { ok: false; error: E };

type FareError =
  | { code: 'UNKNOWN_ROUTE'; route: string }
  | { code: 'SOLD_OUT'; date: string };

function quoteFare(route: string): Result<number, FareError> {
  if (route === 'DAC-CXB') {
    return { ok: true, value: 4_500 };
  }
  return { ok: false, error: { code: 'UNKNOWN_ROUTE', route } };
}

function message(error: FareError): string {
  switch (error.code) {
    case 'UNKNOWN_ROUTE':
      return `${error.route} পথ নেই`;
    case 'SOLD_OUT':
      return `${error.date}-এ আসন নেই`;
  }
}

for (const route of ['DAC-CXB', 'DAC-XYZ']) {
  const result = quoteFare(route);
  console.log(result.ok ? `৳${result.value}` : message(result.error));
}
```

Output

```
৳4500
DAC-XYZ পথ নেই
```

`result.ok` পরীক্ষার পর TypeScript জানে কোন শাখায় `value` আর কোন শাখায় `error` আছে; ভুল Field পড়লে Compile Error। `message`-এর `switch`-এ প্রতিটি `code`-এর ভিন্ন Field (`route`, `date`) নিরাপদে পড়া গেছে।

এখন একটি নতুন Error যোগ করা হলো (`PRICE_CHANGED`), কিন্তু কেউ `message` হালনাগাদ করতে ভুলে গেলেন। এই ভুল ধরতে `default` -এ একটি `never` Check:

exhaustive.ts · TypeScript — চলবে: Node.js

```ts
type FareError =
  | { code: 'UNKNOWN_ROUTE'; route: string }
  | { code: 'SOLD_OUT'; date: string }
  | { code: 'PRICE_CHANGED'; newPrice: number };

function assertNever(value: never): never {
  throw new Error(`অপ্রত্যাশিত: ${JSON.stringify(value)}`);
}

function message(error: FareError): string {
  switch (error.code) {
    case 'UNKNOWN_ROUTE':
      return `${error.route} পথ নেই`;
    case 'SOLD_OUT':
      return `${error.date}-এ আসন নেই`;
    default:
      return assertNever(error);
  }
}

console.log(message({ code: 'SOLD_OUT', date: '2026-12-16' }));
```

Error Message

```
error TS2345: Argument of type '{ code: "PRICE_CHANGED"; newPrice: number; }'
is not assignable to parameter of type 'never'.
```

সব `case` সামলানো থাকলে `default` -এ `error` -এর Type `never` (আর কোনো সম্ভাবনা নেই), এবং `assertNever` সন্তুষ্ট। একটি `case` বাদ পড়লে বাকি সদস্যটি `never` -এ বসানো যায় না—Compiler ঠিক বলে দেয় কোনটি বাদ পড়েছে। নতুন Status বা Error যোগ করার পর এটিই "কোথায় কোথায় হালনাগাদ করতে হবে"-র তালিকা।

২৪.৮ Generics: একবার লেখা, অনেক Type-এ

Generic Function একটি **Type Parameter** নেয়, যাতে একই Code বিভিন্ন Type-এ কাজ করে, এবং Type-এর তথ্য হারায় না:

generics.ts · TypeScript — চলবে: Node.js

```ts
function groupBy<T, K extends PropertyKey>(
  items: readonly T[], key: (item: T) => K,
) {
```

```
  const groups = {} as Record<K, T[]>;
  for (const item of items) {
    (groups[key(item)] ??= []).push(item);
  }
  return groups;
}

const tours = [
  { title: 'Sundarbans', days: 3, region: 'Khulna' },
  { title: 'Ratargul', days: 1, region: 'Sylhet' },
  { title: 'Jaflong', days: 1, region: 'Sylhet' },
];

const byRegion = groupBy(tours, (tour) => tour.region);
const byLength = groupBy(tours, (tour) => (tour.days > 1 ? 'long' : 'short'));
console.log(Object.keys(byRegion), byLength.short?.map((t) => t.title));
```

Output

```
[ 'Khulna', 'Sylhet' ] [ 'Ratargul', 'Jaflong' ]
```

`T` অনুমিত হয়েছে Tour-এর আকার থেকে, `K` হয়েছে `string` এবং দ্বিতীয় ডাকে `'long' | 'short'`। `K extends PropertyKey` একটি **Constraint**: Key হতে পারে শুধু String, Number বা Symbol। Generic Class-এর উদাহরণ Project-এর `MemoryRepository<T, K>`: যেকোনো Type-এর Repository, যার একটি `id` আছে।

২৪.৯ `keyof` এবং Utility Type

বিদ্যমান Type থেকে নতুন Type তৈরি করা যায়, ফলে একই আকার দুবার লিখতে হয় না, এবং মূলটি বদলালে সব আপনাআপনি বদলায়।

Utility	ফল	ব্যবহার
`keyof T`	T-এর সব Key-এর Union	নিরাপদ Field-এর নাম
`Pick<T, K>`	শুধু কয়েকটি Field	নতুন তৈরির ইনপুট
`Omit<T, K>`	কয়েকটি বাদে সব	সারাংশ, Public রূপ
`Partial<T>`	সব Field ঐচ্ছিক	আংশিক হালনাগাদ
`Required<T>`	সব Field বাধ্যতামূলক	Default পূরণের পর
`Readonly<T>`	সব Field অপরিবর্তনীয়	Immutable তথ্য
`Record<K, V>`	প্রতিটি K-এর জন্য একটি V	গণনা, Lookup টেবিল
`ReturnType<F>`	Function-এর ফেরত Type	Factory-র Type

keyof.ts · TypeScript — চলবে: Node.js

```ts
interface Tour {
  title: string;
  price: number;
  days: number;
}

function sortBy<T>(items: readonly T[], key: keyof T): T[] {
  const compare = (a: T, b: T) =>
    (a[key] < b[key] ? -1 : a[key] > b[key] ? 1 : 0);
  return [...items].sort(compare);
}

function update<T>(item: T, changes: Partial<T>): T {
  return { ...item, ...changes };
}

const tours: Tour[] = [
  { title: 'Sajek', price: 6_500, days: 2 },
  { title: 'Kuakata', price: 4_000, days: 2 },
];
console.log(sortBy(tours, 'price').map((t) => t.title));
console.log(update(tours[0]!, { price: 7_000 }));
sortBy(tours, 'prize');
```

Error Message

```
error TS2345: Argument of type '"prize"' is not assignable to parameter
of type 'keyof Tour'.
```

শেষ লাইনের বানান ভুল (`prize`) Compile-এ ধরা পড়েছে, কারণ `key`-এর Type `keyof Tour` = `'title' | 'price' | 'days'`। (ভুল লাইনটি সরালে বাকি Code চলে এবং দাম অনুযায়ী `['Kuakata', 'Sajek']` ছাপে।) Project-এ `NewClaim = Pick<Claim, 'employee' | 'items'>`, `ClaimSummary = Omit<Claim, 'items'> & { itemCount: number }` এবং `StatusCounts = Record<ClaimStatus, number>`।

২৪.১০ Branded Type: একই Type, ভিন্ন অর্থ

টাকার পরিমাণ (পয়সায়), রাতের সংখ্যা, মানুষের সংখ্যা—সবই `number`। কেউ ভুল করে পয়সার জায়গায় টাকা বা রাতের সংখ্যা পাঠালে সাধারণ Type ধরতে পারে না। **Branded Type** একটি অদৃশ্য চিহ্ন যোগ করে: Compile-এর সময় `Paisa` আর সাধারণ `number` আলাদা, Runtime-এ দুটিই সাধারণ সংখ্যা।

brand.ts · TypeScript — চলবে: Node.js

```ts
declare const brand: unique symbol;
type Paisa = number & { readonly [brand]: 'Paisa' };
```

```
function toPaisa(value: number): Paisa {
  if (!Number.isSafeInteger(value) || value < 0) {
    throw new RangeError(`অবৈধ পরিমাণ: ${value}`);
  }
  return value as Paisa;
}

function addFee(amount: Paisa, fee: Paisa): Paisa {
  return toPaisa(amount + fee);
}

const nights = 3;
console.log(addFee(toPaisa(120_000), toPaisa(5_000)));
console.log(addFee(toPaisa(120_000), nights));
```

Error Message

```
error TS2345: Argument of type 'number' is not assignable to parameter
of type 'Paisa'.
```

`Paisa` তৈরির একমাত্র দরজা `toPaisa`, যা Runtime-এ যাচাই করে—তাই একটি `Paisa` মান সব সময় যাচাই করা। এখানে `as`-এর ব্যবহার বৈধ, কারণ তার ঠিক আগের লাইনগুলোই প্রমাণ দিয়েছে। Project-এ `Paisa` ও `ClaimId` দুটিই Branded।

২৪.১১ পেশাদার Workflow: JavaScript থেকে ধাপে ধাপে TypeScript

বাস্তবে নতুন Project-এর চেয়ে বেশি দেখা যায় একটি চালু JavaScript Project, যাকে TypeScript-এ আনতে হবে। পুরোটা একবারে নতুন করে লেখা (Big-bang Rewrite) ঝুঁকিপূর্ণ: মাসের পর মাস দুটি সংস্করণ, এবং নতুন ভুল। নিরাপদ পথটি ধাপে ধাপে, প্রতিটি ধাপের শেষে Project চালু ও পরীক্ষিত।

প্রথম ধাপ: JavaScript-এই Type Check। একটি `.js` File-এর শুরুতে `// @ts-check` লিখলে, এবং JSDoc মন্তব্যে Type বর্ণনা করলে, TypeScript সেই JavaScript-ই যাচাই করে—কোনো File-এর নাম বা Build না বদলে:

discount.js · JavaScript — চলবে: Node.js

```
// @ts-check

/**
 * @param {number} price
 * @param {number} percent
 * @returns {number}
 */
export function discount(price, percent) {
  return Math.round(price * (1 - percent / 100));
}

/** @type {{ title: string, days: number }} */
const tour = { title: 'Sajek', days: '2' };
console.log(discount('500', 10), tour);
```

`tsc --allowJs --checkJs --noEmit` চালালে:

Error Message

```
error TS2322: Type 'string' is not assignable to type 'number'.
error TS2345: Argument of type 'string' is not assignable to parameter
of type 'number'.
```

এই File এখনো সাধারণ JavaScript, Browser বা Node.js-এ যেমন ছিল তেমনই চলে; কিন্তু দুটি ভুল— `days` -এ লেখা, আর `discount` -এ লেখা পাঠানো—এখন Editor ও `tsc` দুটিই দেখায়। অনেক দল এই ধাপেই থেকে যায়: Build Step ছাড়াই Type-এর বেশিরভাগ সুবিধা।

দ্বিতীয় ধাপ: File ধরে ধরে `.ts` । `tsconfig.json` -এ `allowJs: true` দিলে `.js` ও `.ts` একই Project-এ মিশে থাকতে পারে। রূপান্তর শুরু করুন "পাতা" থেকে—যে File অন্য কিছু Import করে না (যেমন `money.js`)—তারপর ধীরে ধীরে মূল Logic-এর দিকে। প্রতিটি File রূপান্তরের পর Test চালান।

তৃতীয় ধাপ: কঠোরতা বাড়ানো। পুরোনো Code-এ `strict` একবারে চালু করলে শত শত Error আসতে পারে। একটি একটি করে চালু করুন (`noImplicitAny` , তারপর `strictNullChecks`), এবং যে ভুল এখনই ঠিক করা যাচ্ছে না, সেখানে `any` নয়, বরং `// @ts-expect-error` এবং পাশে কারণ ও Issue-এর নম্বর লিখুন—পরে ভুলটি ঠিক হলে Compiler নিজেই মনে করিয়ে দেবে (`TS2578`)।

বাইরের Library-র Type। বেশিরভাগ আধুনিক npm Package নিজের Type (`.d.ts` File) সঙ্গে দেয়। পুরোনো JavaScript Package-এর Type প্রায়ই আলাদা `@types/...` Package-এ থাকে (যেমন এই অধ্যায়ের `@types/node`)। কোনোটিই না থাকলে নিজের Project-এ একটি ছোট `declare module 'name'` ঘোষণা লিখে শুধু ব্যবহৃত অংশের Type দিন। `.d.ts` File-এ শুধু Type থাকে, কোনো চলমান Code নয়।

দৈনন্দিন Workflow:

কোথায়	কী চলে	কেন
Editor	প্রতিটি টাইপে Type Check	তাৎক্ষণিক প্রতিক্রিয়া
Terminal	`tsc --noEmit --watch`	পুরো Project-এর অবস্থা
CI	`tsc --noEmit`, তারপর Test	Type ভাঙা Code Merge হয় না
Build	`tsc` (বা Bundler) দিয়ে `dist/`	Deploy বা Publish
Code Review	`any`, `as`, `!`, `@ts-expect-error`-এ প্রশ্ন	ফাঁকফোকর নিয়ন্ত্রণ

একটি গুরুত্বপূর্ণ পার্থক্য মনে রাখুন: Vite, esbuild বা Node.js-এর Type Stripping-এর মতো দ্রুত Tool সাধারণত শুধু Type **মুছে** JavaScript বানায়, যাচাই করে না। তাই এগুলো ব্যবহার করলেও Type Check-এর জন্য আলাদা একটি `tsc --noEmit` ধাপ CI-তে রাখতেই হবে।

২৪.১২ Project: TypeSafe Claims

TypeSafe Claims একটি ভ্রমণ সংস্থার কর্মীদের খরচ-দাবির (Expense Claim) Service: কর্মী হোটেল, যাতায়াত, খাবার ও Visa Fee-র খরচ জমা দেন; নীতির সীমা যাচাই হয়; দাবিটি খসড়া থেকে জমা, অনুমোদন বা প্রত্যাখ্যান এবং পরিশোধের পথে এগোয়। Booking নয়—v30-এর Booking API-র বদলে এই Domain বেছে নেওয়া হয়েছে আগের অধ্যায়ের Booking Project-গুলোর পুনরাবৃত্তি এড়াতে।

> **নোট:** শিক্ষামূলক Demo। নীতির সীমা কাল্পনিক; তথ্য Memory-তে; `api.ts` একটি HTTP-ধাঁচের Adapter, আসল Network Server নয় (Server অধ্যায় ২৫-এ)।

File-এর কাঠামো

Folder-এর কাঠামো

```
typesafe-claims/
├── package.json, tsconfig.json
├── src/
│   ├── types.ts        Brand, ClaimItem Union, Claim, Utility Type, Result
│   ├── money.ts        toPaisa, sum, formatTaka
│   ├── policy.ts       assertNever, নীতির সীমা, বর্ণনা
│   ├── parse.ts        unknown থেকে NewClaim (Type Guard)
│   ├── workflow.ts     State Machine, Status-এর লেবেল
│   ├── repository.ts   Generic Repository
│   ├── service.ts      Use Case, ServiceError Union
│   ├── api.ts          HTTP-ধাঁচের Adapter
│   └── demo.ts
└── tests/              parse, service, api (Runtime);
                        types.check.ts (Compile-time)
```

src/types.ts ও src/money.ts

types.ts · TypeScript — চলবে: Node.js

```ts
// Branded Type: Compile-এর সময় সাধারণ number থেকে আলাদা, Runtime-এ number-ই।
declare const brand: unique symbol;
export type Brand<T, Name extends string> = T & { readonly [brand]: Name };

export type Paisa = Brand<number, 'Paisa'>;
export type ClaimId = Brand<string, 'ClaimId'>;

// প্রতিটি খরচের ধরন নিজস্ব Field-সহ: kind দেখে TypeScript বাকিটা জানে।
export type ClaimItem =
  | { kind: 'hotel'; city: string; nights: number; amount: Paisa }
  | { kind: 'transport'; mode: 'bus' | 'train' | 'air'; amount: Paisa }
  | { kind: 'meal'; people: number; amount: Paisa }
  | { kind: 'visa'; country: string; amount: Paisa };

export type ItemKind = ClaimItem['kind'];
export type ClaimStatus =
  'draft' | 'submitted' | 'approved' | 'rejected' | 'paid';

export interface Claim {
  readonly id: ClaimId;
  readonly employee: string;
  readonly items: readonly ClaimItem[];
  readonly status: ClaimStatus;
  readonly total: Paisa;
}

// Utility Type: বিদ্যমান Type থেকে নতুন Type, নকল ছাড়াই।
export type NewClaim = Pick<Claim, 'employee' | 'items'>;
export type ClaimSummary = Omit<Claim, 'items'> & { itemCount: number };
export type StatusCounts = Record<ClaimStatus, number>;

export type Result<T, E> =
  | { readonly ok: true; readonly value: T }
  | { readonly ok: false; readonly error: E };

export const ok = <T>(value: T): Result<T, never> => ({ ok: true, value });
export const fail = <E>(error: E): Result<never, E> => ({ ok: false, error });
```

money.ts · TypeScript — চলবে: Node.js

```ts
import type { Paisa } from './types.js';

// Branded মান তৈরির একমাত্র দরজা: Runtime যাচাই পেরোলে তবেই Paisa।
export function toPaisa(value: number): Paisa {
  if (!Number.isSafeInteger(value) || value < 0) {
    throw new RangeError(`অবৈধ পরিমাণ: ${value}`);
```

```
  }
  return value as Paisa;
}

export const sum = (values: readonly Paisa[]): Paisa =>
  toPaisa(values.reduce((total, value) => total + value, 0));

export function formatTaka(value: Paisa): string {
  const whole = Math.floor(value / 100).toLocaleString('en-US');
  return `৳${whole}.${String(value % 100).padStart(2, '0')}`;
}
```

`ClaimItem` চার সদস্যের একটি Discriminated Union: `kind: 'hotel'` হলে `city` ও `nights` আছে, `kind: 'meal'` হলে `people`। `ItemKind = ClaimItem['kind']` Union থেকে Discriminant-গুলোর তালিকা বের করে—হাতে আবার লিখতে হয় না। `ok` ও `fail` Generic Helper: `fail` ফেরত দেয় `Result<never, E>`, যা যেকোনো `Result<T, E>`-এ খাপ খায়।

src/policy.ts ও src/workflow.ts

policy.ts · TypeScript — চলবে: Node.js

```
import type { ClaimItem, Paisa } from './types.js';
import { toPaisa } from './money.js';

// কোনো case বাদ পড়লে এখানে Compile Error: item তখন never নয়।
export function assertNever(value: never): never {
  throw new Error(`অপ্রত্যাশিত মান: ${JSON.stringify(value)}`);
}

// প্রতিটি খরচের সর্বোচ্চ অনুমোদিত পরিমাণ (কাল্পনিক নীতি)।
export function limitFor(item: ClaimItem): Paisa {
  switch (item.kind) {
    case 'hotel':
      return toPaisa(item.nights * 8_000_00);
    case 'transport':
      return toPaisa(item.mode === 'air' ? 25_000_00 : 3_000_00);
    case 'meal':
      return toPaisa(item.people * 1_000_00);
    case 'visa':
      return toPaisa(15_000_00);
    default:
      return assertNever(item);
  }
}

export function describe(item: ClaimItem): string {
  switch (item.kind) {
    case 'hotel':
```

```
      return `${item.city}-এ ${item.nights} রাত হোটেল`;
    case 'transport':
      return `${item.mode} যাতায়াত`;
    case 'meal':
      return `${item.people} জনের খাবার`;
    case 'visa':
      return `${item.country} Visa Fee`;
    default:
      return assertNever(item);
  }
}
```

workflow.ts · TypeScript — চলবে: Node.js

```
import type { ClaimStatus } from './types.js';
import { assertNever } from './policy.js';

export type Action = 'submit' | 'approve' | 'reject' | 'pay';

// Record<ClaimStatus, ...>: একটি নতুন Status যোগ করলে এখানে না লিখলে Compile Error।
const MOVES: Record<ClaimStatus, Partial<Record<Action, ClaimStatus>>> = {
  draft: { submit: 'submitted' },
  submitted: { approve: 'approved', reject: 'rejected' },
  approved: { pay: 'paid' },
  rejected: {},
  paid: {},
};

export function nextStatus(
  from: ClaimStatus, action: Action,
): ClaimStatus | null {
  return MOVES[from][action] ?? null;
}

export function statusLabel(status: ClaimStatus): string {
  switch (status) {
    case 'draft':
      return 'খসড়া';
    case 'submitted':
      return 'জমা দেওয়া';
    case 'approved':
      return 'অনুমোদিত';
    case 'rejected':
      return 'প্রত্যাখ্যাত';
    case 'paid':
      return 'পরিশোধিত';
    default:
      return assertNever(status);
```

```
  }
}
```

প্রতিটি `switch`-এর `default`-এ `assertNever`: একটি নতুন খরচের ধরন (যেমন `'insurance'`) যোগ করলে `limitFor` ও `describe` দুটিতেই Compile Error, যতক্ষণ না নতুন `case` লেখা হয়। `MOVES`-এর Type `Record<ClaimStatus, ...>`—নতুন Status যোগ করলে এই টেবিলেও Compile Error। অধ্যায় ২১-এর State Machine এখন Compiler-এর পাহারায়।

src/parse.ts: Trust Boundary

parse.ts · TypeScript · অংশ ১ / ৩ (Part 1 of 3) · চলবে: Node.js

```ts
import type { ClaimItem, NewClaim, Result } from './types.js';
import { ok, fail } from './types.js';
import { toPaisa } from './money.js';

export interface Issue {
  readonly path: string;
  readonly message: string;
}

// Type Guard: Runtime যাচাই, এবং সফল হলে TypeScript-কে Type জানায়।
const isRecord = (v: unknown): v is Record<string, unknown> =>
  typeof v === 'object' && v !== null && !Array.isArray(v);
const isText = (v: unknown): v is string =>
  typeof v === 'string' && v.trim().length > 0 && v.length <= 60;
const isCount = (v: unknown, max: number): v is number =>
  Number.isInteger(v) && (v as number) >= 1 && (v as number) <= max;
const isAmount = (v: unknown): v is number =>
  Number.isSafeInteger(v) && (v as number) > 0;
const MODES = ['bus', 'train', 'air'] as const;
const isMode = (v: unknown): v is (typeof MODES)[number] =>
  MODES.some((mode) => mode === v);

// ...
```

parse.ts · TypeScript · অংশ ২ / ৩ (Part 2 of 3) · চলবে: Node.js

```ts
// ...

function parseItem(raw: unknown, at: string, issues: Issue[]): ClaimItem | null {
  const report = (field: string, message: string): null => {
    issues.push({ path: field ? `${at}.${field}` : at, message });
    return null;
  };
  if (!isRecord(raw)) {
    return report('', 'একটি Object প্রয়োজন');
  }
```

```
  const { kind, amount } = raw;
  if (!isAmount(amount)) {
    return report('amount', 'পয়সায় ধনাত্মক পূর্ণসংখ্যা');
  }
  const paisa = toPaisa(amount);
  switch (kind) {
    case 'hotel': {
      const { city, nights } = raw;
      if (!isText(city)) {
        return report('city', 'শহরের নাম প্রয়োজন');
      }
      return isCount(nights, 30)
        ? { kind, city: city.trim(), nights, amount: paisa }
        : report('nights', '১ থেকে ৩০ রাত');
    }
    case 'transport':
      return isMode(raw.mode)
        ? { kind, mode: raw.mode, amount: paisa }
        : report('mode', 'bus, train অথবা air');
    case 'meal':
      return isCount(raw.people, 20)
        ? { kind, people: raw.people, amount: paisa }
        : report('people', '১ থেকে ২০ জন');
    case 'visa':
      return isText(raw.country)
        ? { kind, country: raw.country.trim(), amount: paisa }
        : report('country', 'দেশের নাম প্রয়োজন');
    default:
      return report('kind', 'অজানা খরচের ধরন');
  }
}

// ...
```

parse.ts · TypeScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

// API-র সীমানা: যা আসে তা unknown; যাচাই পেরোলে তবেই NewClaim।
export function parseNewClaim(input: unknown): Result<NewClaim, Issue[]> {
  const issues: Issue[] = [];
  if (!isRecord(input)) {
    return fail([{ path: '', message: 'একটি JSON Object প্রয়োজন' }]);
  }
  const { employee, items } = input;
  if (!isText(employee)) {
    issues.push({ path: 'employee', message: 'কর্মীর নাম প্রয়োজন' });
  }
  if (!Array.isArray(items) || items.length < 1 || items.length > 20) {
```

```
    issues.push({ path: 'items', message: '১ থেকে ২০টি খরচ' });
  }
  const parsed = Array.isArray(items)
    ? items.map((raw, i) => parseItem(raw, `items[${i}]`, issues))
    : [];
  if (issues.length > 0 || !isText(employee)) {
    return fail(issues);
  }
  const valid = parsed.filter((item) => item !== null);
  return ok({ employee: employee.trim(), items: valid });
}
```

`parseNewClaim`-এর Parameter `unknown`: এই File-এর বাইরে কোথাও বাইরের তথ্য সরাসরি ব্যবহার হয় না। প্রতিটি Field একটি Type Guard দিয়ে যাচাই হয়, এবং `switch (kind)`-এর প্রতিটি `case`-এ TypeScript জানে `kind` কোন Literal—তাই `{ kind, city, nights, amount }` সঠিক Union সদস্য হিসেবে গৃহীত হয়। নতুন Object শুধু পরিচিত Field দিয়ে তৈরি: ইনপুটে `status: 'paid'` বা `approved: true` পাঠালেও তা কোথাও পৌঁছায় না (অধ্যায় ২০-এর Allowlist)। প্রতিটি ভুল একটি পথসহ (`items[1].mode`), যাতে UI সঠিক ঘরে বার্তা দেখাতে পারে।

src/repository.ts ও src/service.ts

repository.ts · TypeScript — চলবে: Node.js

```
// Generic Repository: যেকোনো Type T, যার একটি id আছে, এবং id-র Type K।
export interface Repository<T extends { readonly id: K }, K> {
  get(id: K): T | undefined;
  save(item: T): void;
  list(): readonly T[];
}

export class MemoryRepository<T extends { readonly id: K }, K>
implements Repository<T, K> {
  readonly #items = new Map<K, T>();

  get(id: K): T | undefined {
    return this.#items.get(id);
  }

  save(item: T): void {
    this.#items.set(item.id, item);
  }

  list(): readonly T[] {
    return [...this.#items.values()];
  }
}
```

service.ts · TypeScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```ts
import type {
  Claim, ClaimId, ClaimSummary, Paisa, Result, StatusCounts,
} from './types.js';
import { ok, fail } from './types.js';
import { sum } from './money.js';
import { limitFor } from './policy.js';
import { parseNewClaim, type Issue } from './parse.js';
import { nextStatus, type Action } from './workflow.js';
import type { Repository } from './repository.js';

// প্রতিটি সম্ভাব্য ব্যর্থতা একটি Type; ডাকা Code সবগুলো সামলাতে বাধ্য।
export type ServiceError =
  | { readonly code: 'VALIDATION'; readonly issues: readonly Issue[] }
  | { readonly code: 'OVER_LIMIT'; readonly index: number; readonly limit: Paisa }
  | { readonly code: 'NOT_FOUND'; readonly id: string }
  | { readonly code: 'INVALID_TRANSITION'; readonly from: Claim['status'];
      readonly action: Action };

export function createClaimService(
  repository: Repository<Claim, ClaimId>,
  newId: () => ClaimId,
) {
  function create(input: unknown): Result<Claim, ServiceError> {
    const parsed = parseNewClaim(input);
    if (!parsed.ok) {
      return fail({ code: 'VALIDATION', issues: parsed.error });
    }
    const { employee, items } = parsed.value;
    for (const [index, item] of items.entries()) {
      const limit = limitFor(item);
      if (item.amount > limit) {
        return fail({ code: 'OVER_LIMIT', index, limit });
      }
    }
    const total = sum(items.map((item) => item.amount));
    const claim: Claim = { id: newId(), employee, items, status: 'draft', total };
    repository.save(claim);
    return ok(claim);
  }

// ...
```

service.ts · TypeScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```ts
// ...

  function act(id: ClaimId, action: Action): Result<Claim, ServiceError> {
```

```
    const claim = repository.get(id);
    if (!claim) {
      return fail({ code: 'NOT_FOUND', id });
    }
    const status = nextStatus(claim.status, action);
    if (!status) {
      return fail({ code: 'INVALID_TRANSITION', from: claim.status, action });
    }
    const updated: Claim = { ...claim, status };
    repository.save(updated);
    return ok(updated);
  }

  function summary(): StatusCounts {
    const counts: StatusCounts = {
      draft: 0, submitted: 0, approved: 0, rejected: 0, paid: 0,
    };
    for (const claim of repository.list()) {
      counts[claim.status] += 1;
    }
    return counts;
  }

  const list = (): ClaimSummary[] => repository.list()
    .map(({ items, ...rest }) => ({ ...rest, itemCount: items.length }));

  return { create, act, summary, list };
}

export type ClaimService = ReturnType<typeof createClaimService>;
```

`ServiceError` চার রকম ব্যর্থতার Union, প্রতিটির নিজস্ব তথ্য। Service কোনো Exception ছোড়ে না; প্রতিটি ফল একটি `Result`, তাই ডাকা Code ব্যর্থতা উপেক্ষা করতে পারে না— `result.value` পড়তে হলে আগে `result.ok` যাচাই করতেই হবে। `ClaimService = ReturnType<typeof createClaimService>`: Factory-র ফেরত Type হাতে না লিখে তৈরি।

src/api.ts

api.ts · TypeScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import type { ClaimId } from './types.js';
import type { ClaimService, ServiceError } from './service.js';
import type { Action } from './workflow.js';
import { assertNever } from './policy.js';

export interface ApiRequest {
  readonly method: string;
  readonly path: string;
```

```
  readonly body?: unknown;
}

export interface ApiResponse {
  readonly status: number;
  readonly body: unknown;
}

// প্রতিটি ServiceError-এর HTTP Status; নতুন Error যোগ করলে Compile Error।
function statusOf(error: ServiceError): number {
  switch (error.code) {
    case 'VALIDATION':
    case 'OVER_LIMIT':
      return 422;
    case 'NOT_FOUND':
      return 404;
    case 'INVALID_TRANSITION':
      return 409;
    default:
      return assertNever(error);
  }
}

// ...
```

api.ts · TypeScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

const ACTIONS: readonly Action[] = ['submit', 'approve', 'reject', 'pay'];
const isAction = (v: string): v is Action => ACTIONS.some((a) => a === v);

// Transport Adapter: HTTP-এর মতো অনুরোধ থেকে Service; আসল Network Server নয়।
export function handle(service: ClaimService, request: ApiRequest): ApiResponse {
  const { method, path, body } = request;
  const action = /^\/claims\/([\w-]+)\/(\w+)$/.exec(path);
  const result = method === 'POST' && path === '/claims'
    ? service.create(body)
    : method === 'POST' && action?.[1] && action[2] && isAction(action[2])
      ? service.act(action[1] as ClaimId, action[2])
      : null;
  if (result === null) {
    return method === 'GET' && path === '/claims'
      ? { status: 200, body: service.list() }
      : { status: 404, body: { error: 'পাওয়া যায়নি' } };
  }
  return result.ok
    ? { status: path === '/claims' ? 201 : 200, body: result.value }
```

```
    : { status: statusOf(result.error), body: result.error };
}
```

`statusOf`-এর Exhaustive `switch` প্রতিটি `ServiceError`-কে একটি HTTP Status দেয়; নতুন Error যোগ করে এখানে ভুলে গেলে Compile Error। `isAction` URL-এর লেখাকে Type Guard দিয়ে `Action`-এ রূপান্তর করে। একমাত্র `as ClaimId` এখানে, কারণ Repository-তে খোঁজাই সেই ID-র যাচাই।

src/demo.ts

demo.ts · TypeScript — চলবে: Node.js

```ts
import type { ClaimId } from './types.js';
import { createClaimService } from './service.js';
import { MemoryRepository } from './repository.js';
import { handle } from './api.js';
import { formatTaka } from './money.js';
import { statusLabel } from './workflow.js';
import type { Claim } from './types.js';

let counter = 0;
const newId = () => `EC-${String(++counter).padStart(3, '0')}` as ClaimId;
const service = createClaimService(new MemoryRepository<Claim, ClaimId>(), newId);

// JSON.parse-এর ফল unknown-এর মতোই: বাইরের তথ্য, যাচাই না হওয়া পর্যন্ত।
const body: unknown = JSON.parse(`{
  "employee": "Mina Akter",
  "items": [
    { "kind": "hotel", "city": "Cox's Bazar", "nights": 2, "amount": 1200000 },
    { "kind": "transport", "mode": "bus", "amount": 180000 }
  ]
}`);

const created = handle(service, { method: 'POST', path: '/claims', body });
console.log(created.status, JSON.stringify(created.body).slice(0, 44));

const result = service.act('EC-001' as ClaimId, 'submit');
if (result.ok) {
  const { id, status, total } = result.value;
  console.log(id, statusLabel(status), formatTaka(total));
}

const bad = handle(service, {
  method: 'POST', path: '/claims',
  body: { employee: '', items: [{ kind: 'meal', people: 0, amount: 5 }] },
});
console.log(bad.status, JSON.stringify(bad.body));
```

```
const twice = handle(service, { method: 'POST', path: '/claims/EC-001/pay' });
console.log(twice.status, JSON.stringify(twice.body));
```

Terminal

```
npm run demo
```

Terminal

```
201 {"id":"EC-001","employee":"Mina Akter","item
EC-001 জমা দেওয়া ৳13,800.00
422 {"code":"VALIDATION","issues":[{"path":"employee","message":"কর্মীর নাম প্রয়োজন"},
    {"path":"items[0].people","message":"১ থেকে ২০ জন"}]}
409 {"code":"INVALID_TRANSITION","from":"submitted","action":"pay"}
```

এটি Compile করা `dist/src/demo.js` চালানোর আসল Output (এই বইয়ের QA-তে)। প্রথম লাইন সংক্ষিপ্ত (`slice(0, 44)`)।

Test: Compile-time ও Runtime

types.check.ts · TypeScript — চলবে: Node.js

```
// Compile-time Test: এই File কিছু চালায় না। প্রতিটি @ts-expect-error-এর পরের
// লাইনে Type Error থাকতেই হবে; না থাকলে tsc নিজেই ব্যর্থ হয়। অর্থাৎ Type-এর
// নিশ্চয়তাগুলো (Brand, Union, readonly) কেউ ভুল করে ঢিলে করলে Build ভাঙবে।
import type {
  Claim, ClaimId, ClaimItem, Paisa, StatusCounts,
} from '../src/types.js';
import { toPaisa } from '../src/money.js';

export function compileTimeChecks(claim: Claim): void {
  // @ts-expect-error সাধারণ number সরাসরি Paisa নয়; toPaisa দিয়ে যাচাই করতে হয়।
  const raw: Paisa = 500;

  // @ts-expect-error সাধারণ string ClaimId নয়।
  const id: ClaimId = 'EC-1';

  // @ts-expect-error hotel-এর nights বাধ্যতামূলক।
  const hotel: ClaimItem = { kind: 'hotel', city: 'Sylhet', amount: toPaisa(1) };

  const meal: ClaimItem = {
    kind: 'meal', people: 2, amount: toPaisa(1),
    // @ts-expect-error meal-এ city নেই (Union-এর ভুল সদস্য)।
    city: 'x',
  };

  // @ts-expect-error readonly Field বদলানো যায় না।
```

```
  claim.status = 'paid';

  // @ts-expect-error Record<ClaimStatus, number>-এ প্রতিটি অবস্থা লাগবে।
  const counts: StatusCounts = { draft: 0, submitted: 0 };

  void [raw, id, hotel, meal, counts];
}
```

parse.test.ts · TypeScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// Runtime Test: TypeScript Runtime-এ নেই, তাই unknown ইনপুটের যাচাই Test করতে হয়।
import test from 'node:test';
import assert from 'node:assert/strict';
import { parseNewClaim } from '../src/parse.js';

// ...

test('প্রতিটি ভুলের পথ ও বার্তা', () => {
  const result = parseNewClaim({
    employee: '',
    items: [
      { kind: 'meal', people: 0, amount: 5 },
      { kind: 'transport', mode: 'rocket', amount: 10 },
      { kind: 'spa', amount: 10 },
      { kind: 'visa', country: 'Japan', amount: -1 },
      'text',
    ],
  });
  assert.equal(result.ok, false);
  if (!result.ok) {
    assert.deepEqual(result.error.map((i) => i.path), [
      'employee', 'items[0].people', 'items[1].mode', 'items[2].kind',
      'items[3].amount', 'items[4]',
    ]);
  }
});

// ...
```

দুই ধরনের Test, দুই ধরনের নিশ্চয়তা। `types.check.ts` কিছুই চালায় না; এর প্রতিটি `@ts-expect-error`-এর পরের লাইনে Type Error থাকতেই হবে, নইলে `tsc` নিজেই "Unused '@ts-expect-error' directive" বলে ব্যর্থ হয়। এই বইয়ের QA-তে এটি যাচাই করা হয়েছে: একটি লাইন ঠিক করে দিলে (`toPaisa(500)`) Build ভেঙেছে `error TS2578`-এ। অর্থাৎ কেউ ভবিষ্যতে `Paisa`-কে সাধারণ `number` বানিয়ে দিলে Build ব্যর্থ হবে। আর `parse.test.ts` Runtime-এ যাচাই করে যে প্রতিটি অবৈধ ইনপুট ধরা পড়ে—কারণ Type Runtime-এ নেই, এবং বাইরের তথ্য Type মানে না।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd typesafe-claims
npm install
npm run typecheck
npm test
```

`npm install` TypeScript ও `@types/node` আনে (এই বইয়ের QA-তে Install করা হয়নি; উপরের নোট দেখুন)। `npm run typecheck` (`tsc --noEmit`) শুধু Type যাচাই করে। `npm test` আগে `tsc` দিয়ে `dist/`-এ Build করে (এই ধাপে `types.check.ts`-এর Compile-time Test-ও চলে), তারপর `node --test "dist/tests/*.test.js"` ১১টি Runtime Test চালায়: `parse` ৪, `service` ৫, `api` ২; শেষে `# pass 11`, `# fail 0`।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`Cannot find module './money.js'` (Compile)	Import-এ `.ts` বা Extension নেই	`.js` লিখুন (NodeNext)
`Cannot find module 'node:test'`	`@types/node` নেই	`npm install --save-dev @types/node`
Test File খুঁজে পায় না	Build হয়নি, বা `.ts` চালানোর চেষ্টা	`npm test` (আগে `tsc`)
`Unused '@ts-expect-error'`	একটি Type-এর নিশ্চয়তা ঢিলে হয়েছে	Type পরিবর্তনটি পুনর্বিবেচনা
`is possibly 'undefined'`	`noUncheckedIndexedAccess`	যাচাই বা `?.`

২৪.১৩ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
`any` দিয়ে Error চাপা	Type Check বন্ধ, ভুল Runtime-এ	`unknown` ও Type Guard
`JSON.parse(x) as T`	যাচাই ছাড়া বিশ্বাস	Runtime Validation
`strict` বন্ধ	`null`-এর ভুল ধরা পড়ে না	নতুন Project-এ সব সময় চালু
`!` দিয়ে `undefined` উপেক্ষা	Runtime Error থেকে যায়	Narrowing, `?.`, `??`
`switch`-এ `default: return ''`	নতুন সদস্যের জন্য নীরব ভুল	`assertNever`
সব জায়গায় Type লেখা	অপ্রয়োজনীয় শব্দ	Inference; সীমানায় স্পষ্ট Type
একই আকার বারবার লেখা	অমিল হয়ে যায়	`Pick`, `Omit`, `Partial`

ভুল	কেন সমস্যা	সমাধান
ধরে নেওয়া TypeScript ইনপুট নিরাপদ করে	Runtime-এ Type নেই	Server-এ যাচাই, অধ্যায় ২০
`strip-types`-কে Type Check ভাবা	শুধু মুছে ফেলে	CI-তে `tsc --noEmit`

২৪.১৪ Debugging নির্দেশিকা

১. **Error-এর Code পড়ুন:** `TS2345` (ভুল Argument), `TS2322` (ভুল মান বসানো), `TS18048` (সম্ভাব্য `undefined`), `TS18046` (`unknown`), `TS2578` (অপ্রয়োজনীয় `@ts-expect-error`)।

২. **Editor-এ Hover করুন:** প্রতিটি Variable-এর অনুমিত Type দেখায়; প্রত্যাশার সঙ্গে না মিললে সেখানেই ভুল।

৩. **দীর্ঘ Error-এর শেষ লাইন** সাধারণত আসল কারণ বলে।

৪. **Runtime Error কিন্তু Compile ঠিক:** কোথাও `any`, `as` বা `!`, অথবা বাইরের তথ্য যাচাই হয়নি।

৫. **`tsc --noEmit --watch`:** প্রতিটি সংরক্ষণে Type Check।

৬. **Source Map** (`sourceMap: true`): Compile করা JavaScript-এর Error Stack মূল `.ts` লাইনে নিয়ে যায়।

২৪.১৫ পেশাদার অভ্যাস

- **`strict`** চালু, এবং **`noUncheckedIndexedAccess`**-ও।
- **সীমানায় `unknown`, ভেতরে নির্দিষ্ট Type;** যাচাই এক জায়গায়।
- **`any`, `as`** ও **`!`** **Code Review-তে প্রশ্ন তোলার মতো;** প্রতিটির পাশে কারণ লিখুন।
- **Status, Mode, Category-র জন্য Literal Union,** `switch`-এ `assertNever`।
- **Error-এর জন্য Discriminated Union,** যাতে ডাকা Code প্রতিটি ক্ষেত্র সামলায়।
- **Type বারবার লিখবেন না:** Utility Type ও `ReturnType`।
- **CI-তে `tsc --noEmit`,** Test-এর আগে।
- **পুরোনো JavaScript Project ধাপে ধাপে:** `allowJs`, তারপর একটি একটি File।

২৪.১৬ অনুশীলনী

প্রাথমিক স্তর

১. `function discount(price: number, percent: number): number` লিখুন, এবং দেখান `discount('500', 10)` Compile Error দেয়।

২. `type Seat = 'window' | 'aisle' | 'middle'`-এর একটি Variable-এ `'floor'` বসালে কী ঘটে? কেন?

৩. `const` ও `let`-এ Inference-এর পার্থক্য উদাহরণসহ লিখুন।

৪. `users.find(...)`-এর ফল নিরাপদে ব্যবহারের তিনটি উপায় লিখুন।

৫. `any` ও `unknown`-এর একটি পার্থক্য এক বাক্যে।

মধ্যম স্তর

৬. `isTour(value: unknown): value is Tour` Type Guard লিখুন (`title: string`, `days: number`), এবং দুটি JSON-এ পরীক্ষা করুন।

৭. একটি `Shape` Discriminated Union (`circle`, `rectangle`) ও Exhaustive `area` Function লিখুন।

৮. `pluck<T, K extends keyof T>(items: T[], key: K): T[K][]` লিখুন।

৯. TypeSafe Claims-এ একটি নতুন খরচের ধরন `insurance` (Field: `provider`) যোগ করলে কোন কোন File-এ Compile Error হবে? তালিকা করুন।

চ্যালেঞ্জ স্তর

১০. `ok` / `fail` দিয়ে একটি `parseAge(input: unknown): Result<number, string>` লিখুন এবং ফলাফল সামলান।

১১. একটি Generic `LookupCache<K, V>` Class লিখুন, যার `get` ফেরত দেয় `V | undefined`, এবং `getOr(key, fallback)`।

১২. `Readonly` ও `as const` দিয়ে একটি অপরিবর্তনীয় Config Object তৈরি করুন, এবং দেখান যে তার মান বদলানো Compile Error।

২৪.১৭ অনুশীলনীর সমাধান

সমাধান ১

discount.ts · TypeScript — চলবে: Node.js

```ts
function discount(price: number, percent: number): number {
  return Math.round(price * (1 - percent / 100));
}

console.log(discount(500, 10));
console.log(discount('500', 10));
```

Error Message

```
error TS2345: Argument of type 'string' is not assignable to parameter
of type 'number'.
```

সমাধান ২ — Compile Error (`TS2322`): `'floor'` Union-এর তিনটি Literal-এর কোনোটি নয়। Union মানে "শুধু এগুলো", তাই বানান ভুল বা অজানা মান Code চালানোর আগেই ধরা পড়ে।

সমাধান ৩ — `let count = 5` -এর Type `number` (পরে বদলাতে পারে), `const count = 5` -এর Type Literal `5` । তেমনি `let mode = 'bus'` হলো `string` , `const mode = 'bus'` হলো `'bus'` । তাই `Mode` Union-এ পাঠাতে `const` কাজ করে, `let` করে না (যদি না `let mode: Mode = 'bus'` লেখা হয়)।

সমাধান ৪ — (ক) `if (user) { ... }` দিয়ে Narrowing; (খ) `user?.name ?? 'অজানা'` ; (গ) না পেলে স্পষ্ট Error: `if (!user) throw new Error('পাওয়া যায়নি')` , তার পরের লাইনে `user` নিশ্চিত। `user!.name` এড়িয়ে চলুন।

সমাধান ৫ — `any` সব Type Check বন্ধ করে, আর `unknown` যাচাই (Narrowing) না করা পর্যন্ত কিছুই করতে দেয় না।

সমাধান ৬

is-tour.ts · TypeScript — চলবে: Node.js

```ts
interface Tour {
  title: string;
  days: number;
}

function isTour(value: unknown): value is Tour {
  if (typeof value !== 'object' || value === null) {
    return false;
  }
  const record = value as Record<string, unknown>;
  return typeof record.title === 'string'
    && Number.isInteger(record.days) && (record.days as number) > 0;
}

const inputs = ['{ "title": "Sajek", "days": 2 }',
  '{ "title": "Sajek", "days": "2" }'];
for (const text of inputs) {
  const data: unknown = JSON.parse(text);
  console.log(isTour(data) ? `${data.title}: ${data.days} দিন` : 'অবৈধ');
}
```

```
}
```

Output

```
Sajek: 2 দিন
অবৈধ
```

সমাধান ৭

shapes.ts · TypeScript — চলবে: Node.js

```ts
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'rectangle'; width: number; height: number };

function assertNever(value: never): never {
  throw new Error(`অপ্রত্যাশিত: ${JSON.stringify(value)}`);
}

function area(shape: Shape): number {
  switch (shape.kind) {
    case 'circle':
      return Math.PI * shape.radius ** 2;
    case 'rectangle':
      return shape.width * shape.height;
    default:
      return assertNever(shape);
  }
}

const shapes: Shape[] = [
  { kind: 'circle', radius: 1 },
  { kind: 'rectangle', width: 2, height: 3 },
];
console.log(shapes.map((s) => area(s).toFixed(2)));
```

Output

```
[ '3.14', '6.00' ]
```

সমাধান ৮

pluck.ts · TypeScript — চলবে: Node.js

```ts
function pluck<T, K extends keyof T>(items: readonly T[], key: K): T[K][] {
  return items.map((item) => item[key]);
}

const tours = [{ title: 'Sajek', days: 2 }, { title: 'Ratargul', days: 1 }];
const titles: string[] = pluck(tours, 'title');
const days: number[] = pluck(tours, 'days');
console.log(titles, days);
```

Output

```
[ 'Sajek', 'Ratargul' ] [ 2, 1 ]
```

`T[K]` (Indexed Access Type) মানে "T-এর K Field-এর Type": `'title'`-এ `string`, `'days'`-এ `number` — তাই ফলের Type-ও সঠিক।

সমাধান ৯ — `types.ts`-এ Union-এ নতুন সদস্য যোগ করলে: `policy.ts`-এর `limitFor` ও `describe` (দুটি `assertNever`), এবং `parse.ts` —এখানে Compile Error নয়, কারণ অজানা `kind` `default`-এ যায়; তাই নতুন `case` যোগ না করলে Runtime-এ "অজানা খরচের ধরন"। শিক্ষা: Exhaustiveness Compiler-কে তখনই সাহায্য করে, যখন `switch` একটি Union-এর ওপর; `unknown` ইনপুটের `switch`-এর জন্য একটি Runtime Test লাগে (`parse.test.ts`-এ একটি নতুন সারি)।

সমাধান ১০

parse-age.ts · TypeScript — চলবে: Node.js

```ts
type Result<T, E> = { ok: true; value: T } | { ok: false; error: E };
const ok = <T>(value: T): Result<T, never> => ({ ok: true, value });
const fail = <E>(error: E): Result<never, E> => ({ ok: false, error });

function parseAge(input: unknown): Result<number, string> {
  const value = typeof input === 'string' ? Number(input.trim()) : input;
  if (typeof value !== 'number' || !Number.isInteger(value)) {
    return fail('বয়স পূর্ণসংখ্যা হতে হবে');
  }
  return value >= 0 && value <= 120 ? ok(value) : fail('বয়স ০ থেকে ১২০');
}

for (const input of ['34', 34.5, 200, null]) {
  const result = parseAge(input);
  console.log(result.ok ? `বৈধ: ${result.value}` : `ভুল: ${result.error}`);
}
```

Output

```
বৈধ: 34
ভুল: বয়স পূর্ণসংখ্যা হতে হবে
ভুল: বয়স ০ থেকে ১২০
ভুল: বয়স পূর্ণসংখ্যা হতে হবে
```

সমাধান ১১

cache.ts · TypeScript — চলবে: Node.js

```ts
class LookupCache<K, V> {
  readonly #entries = new Map<K, V>();

  set(key: K, value: V): this {
    this.#entries.set(key, value);
    return this;
  }

  get(key: K): V | undefined {
    return this.#entries.get(key);
  }

  getOr(key: K, fallback: V): V {
    return this.#entries.get(key) ?? fallback;
  }
}
```

```
const rates = new LookupCache<string, number>().set('USD-BDT', 121.45);
const rate: number = rates.getOr('EUR-BDT', 0);
console.log(rates.get('USD-BDT'), rate, rates.get('JPY-BDT'));
```

Output

```
121.45 0 undefined
```

`get`-এর ফেরত Type `number | undefined` হওয়ায় Caller-কে না-থাকার ক্ষেত্র ভাবতে হয়; `getOr` সেই সিদ্ধান্ত একবারে নেয়। লক্ষ করুন, `??` দিয়ে সংরক্ষিত `0` বা `''`-ও ঠিক থাকে (যা `||` দিলে হারিয়ে যেত)।

সমাধান ১২

config.ts · TypeScript — চলবে: Node.js

```
const CONFIG = {
  currency: 'BDT',
  limits: { hotelPerNight: 8_000, mealPerPerson: 1_000 },
  modes: ['bus', 'train', 'air'],
} as const;

type TravelMode = (typeof CONFIG.modes)[number];
const mode: TravelMode = 'air';

console.log(mode, CONFIG.limits.hotelPerNight);
CONFIG.limits.hotelPerNight = 20_000;
CONFIG.modes.push('ship');
```

Error Message

```
error TS2540: Cannot assign to 'hotelPerNight' because it is a read-only property.
error TS2339: Property 'push' does not exist on type
'readonly ["bus", "train", "air"]'.
```

`as const` সব স্তরে `readonly` ও Literal Type দেয়, তাই `modes` থেকে `'bus' | 'train' | 'air'` Union-ও তৈরি করা গেছে। মনে রাখুন, এটি শুধু Compile-time: Runtime-এ বদলানো আটকাতে `Object.freeze` লাগে (অধ্যায় ১৮)।

২৪.১৮ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. TypeScript-এর Type Runtime-এ কী হয়?

ক) Browser যাচাই করে

খ) Compile-এর পর মুছে যায় (Type Erasure)

গ) Node.js যাচাই করে

ঘ) ধীর হয়ে যায়

প্রশ্ন ২. Server-এ আসা JSON-এর জন্য কোন Type উপযুক্ত?

ক) `unknown`, তারপর Runtime যাচাই

খ) `any`

গ) `as Customer`

ঘ) `object`

প্রশ্ন ৩. `const mode = 'bus'`-এর অনুমিত Type?

ক) `string`

খ) `any`

গ) Literal `'bus'`

ঘ) `unknown`

প্রশ্ন ৪. `strictNullChecks` কী নিশ্চিত করে?

ক) কোনো মান কখনো `null` নয়

খ) Runtime-এ `null` আটকানো হয়

গ) `null` স্বয়ংক্রিয়ভাবে `0`

ঘ) `undefined` হতে পারে এমন মান যাচাই ছাড়া ব্যবহার করা যায় না

প্রশ্ন ৫. `switch`-এর `default`-এ `assertNever(x)` কেন?

ক) Union-এর কোনো সদস্য বাদ পড়লে Compile Error পেতে

খ) Runtime দ্রুত করতে

গ) `x`-কে `any` করতে

ঘ) Test এড়াতে

> **প্রশ্ন ৬.** Type Guard-এর ফেরত Type কোনটি?
>
> ক) `boolean | Type`
>
> খ) `value is Customer`
>
> গ) `Customer`
>
> ঘ) `never`

> **প্রশ্ন ৭.** `Partial<Tour>` কী?
>
> ক) Tour-এর প্রথম অর্ধেক Field
>
> খ) শুধু `readonly` Field
>
> গ) Tour-এর সব Field ঐচ্ছিক
>
> ঘ) Tour-এর একটি Copy

> **প্রশ্ন ৮.** Branded Type (`Paisa`)-এর লাভ কী?
>
> ক) Runtime-এ পরিমাণ যাচাই
>
> খ) Memory কম
>
> গ) দ্রুত গণনা
>
> ঘ) একই `number` হলেও ভিন্ন অর্থের মান ভুলে মেশানো Compile-এ ধরা

> **প্রশ্ন ৯.** `// @ts-expect-error`-এর পরের লাইনে কোনো Error না থাকলে?
>
> ক) `tsc` ব্যর্থ হয় (Unused directive)
>
> খ) কিছুই না
>
> গ) Runtime Error
>
> ঘ) Warning মাত্র

> **প্রশ্ন ১০.** Node.js-এর `--experimental-strip-types` কী করে?
>
> ক) Type যাচাই করে চালায়
>
> খ) Type মুছে চালায়, যাচাই করে না
>
> গ) TypeScript Install করে
>
> ঘ) JavaScript-কে TypeScript-এ রূপান্তর করে

২৪.১৯ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	তাই Runtime-এ ইনপুট যাচাই আলাদা দায়িত্ব।
২	ক	`any` ও `as` যাচাই ছাড়াই বিশ্বাস করে।
৩	গ	`const` বদলায় না, তাই সবচেয়ে নির্দিষ্ট Type।
৪	ঘ	Compile-time নিশ্চয়তা; Runtime-এ যাচাই আপনার Code-এ।
৫	ক	বাকি সদস্য `never`-এ বসানো যায় না।
৬	খ	Runtime-এ `boolean`, Compiler-এর কাছে Type-এর তথ্য।
৭	গ	আংশিক হালনাগাদের জন্য।
৮	ঘ	Runtime-এ সাধারণ সংখ্যাই; সুবিধাটি Compile-এ।
৯	ক	`TS2578`; তাই এটি Compile-time Test।
১০	খ	Type Check-এর জন্য `tsc` লাগবেই।

২৪.২০ অধ্যায়ের সারসংক্ষেপ

- TypeScript Compile-এর আগে ভুল ধরে, তারপর Type মুছে JavaScript দেয়; Runtime-এ কোনো Type নেই।
- `strict` ও `noUncheckedIndexedAccess` চালু রাখুন; `null` / `undefined` যাচাই বাধ্যতামূলক হয়।
- Inference বেশিরভাগ Type জানে; সীমানায় (Parameter, ফেরত, Export) স্পষ্ট Type।
- Literal Union সীমিত মানের তালিকা; Discriminated Union বিভিন্ন আকারের তথ্য ও Error।
- `assertNever` ও `Record<Union, ...>` দিয়ে সম্পূর্ণতা Compiler-এর পাহারায়।
- বাইরের তথ্য `unknown`, Type Guard দিয়ে যাচাই; `any`, `as`, `!` এড়িয়ে চলুন।
- Generics ও Utility Type: একবার লেখা, Type না হারিয়ে পুনর্ব্যবহার।
- Branded Type একই Runtime Type-এর ভিন্ন অর্থ আলাদা রাখে।
- Compile-time Test (`@ts-expect-error`) Type-এর নিশ্চয়তা পাহারা দেয়; Runtime Test বাইরের তথ্যের যাচাই।

পরের অধ্যায়ে Node.js দিয়ে Backend: REST API, Express, Layered Architecture, Authentication এবং একটি পরীক্ষিত Server।

২৪.২১ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Static Typing	Code চালানোর আগে Type যাচাই।

পরিভাষা	অর্থ
Type Erasure	Compile-এর পর Type মুছে যাওয়া।
Inference	মান দেখে Type অনুমান।
Literal Type	একটি নির্দিষ্ট মানই Type (`'bus'`)।
Union	কয়েকটি Type-এর যেকোনো একটি।
Narrowing	যাচাইয়ের পর Type আরও নির্দিষ্ট হওয়া।
Type Guard	`value is T` ফেরত দেওয়া যাচাই Function।
Discriminated Union	একটি সাধারণ Literal Field-যুক্ত Union।
`never`	যে Type-এর কোনো মান নেই।
Generic	Type Parameter-যুক্ত Function বা Class।
Utility Type	বিদ্যমান Type থেকে নতুন Type (`Pick`, `Partial`)।
Branded Type	অদৃশ্য চিহ্নযুক্ত আলাদা Type।

২৪.২২ আরও পড়ুন

- **TypeScript Handbook:** `https://www.typescriptlang.org/docs/handbook/intro.html` — ভাষার পূর্ণ পরিচয়।
- **TSConfig Reference:** `https://www.typescriptlang.org/tsconfig/` — প্রতিটি Compiler Setting।
- **TypeScript — Narrowing:** `https://www.typescriptlang.org/docs/handbook/2/narrowing.html` — Type Guard ও Discriminated Union।
- **TypeScript — Utility Types:** `https://www.typescriptlang.org/docs/handbook/utility-types.html` — `Pick`, `Omit`, `Partial`।
- **Node.js — TypeScript support:** `https://nodejs.org/api/typescript.html` — Type Stripping ও তার সীমা।